

LLDD BE - Job 8 CreateCompensationDocument

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

สร้างเอกสารประกันรายได้อัตโนมัติ: สร้าง `sgi_compensation_documents` จาก `impact profile` และข้อมูลชดเชยในฐานข้อมูลเดียวกัน แทนการเขียนไฟล์ `BPM06001O` และ SFTP ไป `compensateflow`; ไม่เรียก `workflow` โดยตรง

1.2 ขอบเขตงาน

- Main class/script: `document.service.createFromImpact` / (internal scheduler / service)
- Phase: C
- Output: `sgi_compensation_documents` (DB)
- Estimate: 17 ชั่วโมง
- Argument รับผ่าน INPUT (JSON) — local ใช้ `env JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง `job_configs` และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี @Cron) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	23 ชั่วโมง = implementation 17 + unit test 6 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	<code>ExportImpactStoreFlowToBPM</code> (<code>fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreFlowToBPM.java</code>)
Target repository	<code>SBP/srm-sps-spsap-sop-sgi-batch</code> (NestJS 11 + TypeORM · schema <code>sps_store</code> · มติ 2026-09-02 — ย้ายมาจาก <code>store-backend</code>) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน <code>src/main.ts</code> แล้วรับ argument ผ่าน <code>JOB_NAME/INPUT</code> (local) หรือ <code>argv[3]/argv[2]</code> (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ @Cron · ดู <code>srm sps spsap sop sgi batch</code>

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา `request/response` หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	<code>LLDD-BE-API-Common-Contracts.pdf</code>	<code>envelope {success, data}</code> · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	<code>LLDD-BE-Database-Structure.pdf</code>	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	<code>LLDD-BE-Integration-SBP-Platform.pdf</code>	header จาก BFF (<code>x-api-key / x-user - *</code>) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	<code>LLDD-BE-API-Common-Contracts.pdf</code>	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	<code>LLDD-BE-Job-6-ExportImpactStoreToSTA.pdf</code>	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-create-compensation-document` ใน `src/main.ts` ของ SBP/srm-sps-spsap-sop-sgi-batch โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรับ	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	<code>env JOB_NAME</code>	<code>env INPUT</code> (JSON string)	<code>JOB_NAME=sgi-create-compensation-document INPUT='{\"impactMonth\":\"2026-06\"}' npm run start</code>
AWS Batch (ตารางเวลาจริง)	<code>process.argv[3]</code>	<code>process.argv[2]</code> (JSON string)	<code>node dist/main.js '{\"impactMonth\":\"2026-06\"}' sgi-create-compensation-document</code>

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron/@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นโยบายของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`'INPUT'` เป็น JSON object · ไม่ส่ง = `'{}'`)

ระบบเดิมรับ argument แบบนี้: ไม่รับ args

ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างดี — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วทำอะไร (default)	Validation	ตรงกับ argument เดิม
<code>impactMonth</code>	string	งวดล่าสุดที่พร้อม	YYYY-MM	ของใหม่
<code>impactProcessIds</code>	number[]	null = ทุกรอบที่เข้าเงื่อนไข	ต้องมีอยู่จริงใน <code>sgi_fgi_impact_processes</code>	ของใหม่
<code>dryRun</code>	boolean	false	true = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
<code>limit</code>	number	null	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกากการ validate ที่ทุก job ต้องทำเหมือนกัน — parse INPUT ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default ง่ายๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- ระบุ `impactProcessIds` ไม่ข้ามกฎ `UNIQUE(impact_process_id)` — รอบที่มีเอกสารแล้วต้อง skip พร้อมคืน `doc_no` เดิม

2.2 Input / Output

Stage	Contract for implementation
Input	Impact-store compensation rows in initial status with workflow sequence values and no prior confirm-receive output.
Progress	update BPM sequence, query eligible impact-store rows, refresh not-OPT data, generate workflow payload, insert confirm-receive rows, upload/export, notify.
Output	Impact-store workflow create payload/output with generated sequence numbers and duplicate guard.

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม
2	query impact profile สถานะ I + forecast + ยังไม่สร้างเอกสาร (ใช้ impact_process_id เป็น idempotency key)
3	ข้อมูลผู้อนุมัติ/ร้าน/ยอดชดเชยครบ? No: บันทึก reject reason / ไม่สร้างเอกสาร
4	generate doc_no YYYY/xxxxx (running ต่อปี ค.ศ. (มติ 2026-08-06))
5	insert sgi_compensation_documents (ผูก impact_process_id และสถานะเริ่มต้น)
6	insert sgi_interface_transactions: data_name = IMPACT_STORE · direction = INTERNAL · status = COMPLETED (ไม่สร้างไฟล์ BPM06001O แล้ว — เขียน DB ตรงจึงไม่มี ACK ให้รอ)
7	จบ - workflow เปิดโดย Job 8b / POST /sgi/workflow/instances

2.4 เงื่อนไขตัดสินใจ (Decision Rules) — ตัดสินจากอะไร

Job 8 สร้างเอกสารประกันรายได้ — 2 เรื่องที่ต้องชัดคือ รอบไหนได้เอกสาร และ เลขเอกสารจองอย่างไรไม่ให้ชนกัน

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
รอบชดเชยนี้ได้เอกสารหรือยัง	sgi_fgi_impact_processes .id เทียบกับ sgi_compensation_documents.impact_process_id (UNIQUE)	มีเอกสารของ impact_process_id นั้นอยู่แล้ว = skip และคืน doc_no เดิม	ห้ามสร้างเอกสารใบที่ 2 ให้รอบเดียวกัน — นี่คือกลไกกัน rerun ซ้ำหลักของ job นี้
รอบนี้พร้อมสร้างเอกสารหรือยัง	sgi_fgi_impact_processes .flag_action · sgi_fgi_impact_sales_summaries.sale_status ของรอบเดียวกัน	รอบต้อง active (flag_action IN ('Y', 'W')) และมียอดขาย/Growth ที่คำนวณแล้ว (sales_status = 'Y')	ยังไม่พร้อม = ข้ามรอบนี้ ให้รอบถัดไปหยิบ (ไม่ใช่ error)
เลขเอกสารเป็นอะไร	sgi_document_running_numbers (year · running_no) · UNIQUE(year, running_no)	รูปแบบ YYYY/xxxxx ด้วย ปี ค.ศ. · lock running number ต่อปีภายใน transaction เดียวกับการ insert เอกสาร	ช่องว่างของเลขเป็นเรื่องปกติ (rerun/conflict) — เลขรับประกันแค่ไม่ซ้ำ ไม่รับประกันความต่อเนื่อง และ ห้าม reuse เลขที่จองไปแล้ว

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
รูปแบบเลขเอกสาร	YYYY/xxxxx · ปี ค.ศ.	มติ 2026-08-06 (ทั้งระบบเป็น ค.ศ. ยกเว้นไฟล์ interface)
flag_action ที่ถือว่า active	Y · W	โดเมนเดิมของ FGI_IMPACT_STORE_ON_PROCESS

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
กำหนดการรัน (Cron)	30 17 7-31 **	แก้ไขได้	ใช้รอบเดิม แต่ปลายทางเป็น DB ภายใน
Target table	sgi_compensation_documents	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	สร้าง doc_no YYYY/xxxxx และผูก impact_process_id
เงื่อนไขเลือกข้อมูล	สถานะ I + forecast + ยังไม่สร้างเอกสาร	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	Gen Flow Gate อยู่ที่ Job 8b / Workflow Engine

Field / UI	Format	Validation	Behavior
ข้อห้ามเชิงสถาปัตยกรรม	ห้ามสร้างไฟล์ BPM06001O, ห้าม SFTP, ห้ามเรียก K2 REST	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ใช้ Document Service + DB transaction เท่านั้น

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	Impact-store compensation rows in initial status with workflow sequence values and no prior confirm-receive output.	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	Impact-store workflow create payload/output with generated sequence numbers and duplicate guard.	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	UNIQUE(impact_process_id) และ UNIQUE(year,running_no); lock running number ต่อปีใน transaction; conflict ต้องคืน/อ้าง doc_no เดิม และยอมให้เลขที่จองกระโดดโดยห้าม reuse	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	lock เลขรัน + insert document + update process + tracking (direction=INTERNAL) ใน transaction เดียว	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	internal service account เท่านั้น; ห้ามสร้างไฟล์ BPM06001O, ห้าม SFTP และห้ามเก็บ K2 credential	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreFlowToBPM.java	9-17	Legacy main entrypoint for exporting impact-store flow data.
fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java	518-657	Build impact-store BPM payload, write file, upload, backup, notification.
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	1654-1692	Query impact-store rows eligible for workflow export.

Line ranges refer to the legacy Java implementation under batchjob/fcsJar/ (path นับจากราก sbp-prototype/). Use these ranges to preserve business behavior while implementing the target Node job.

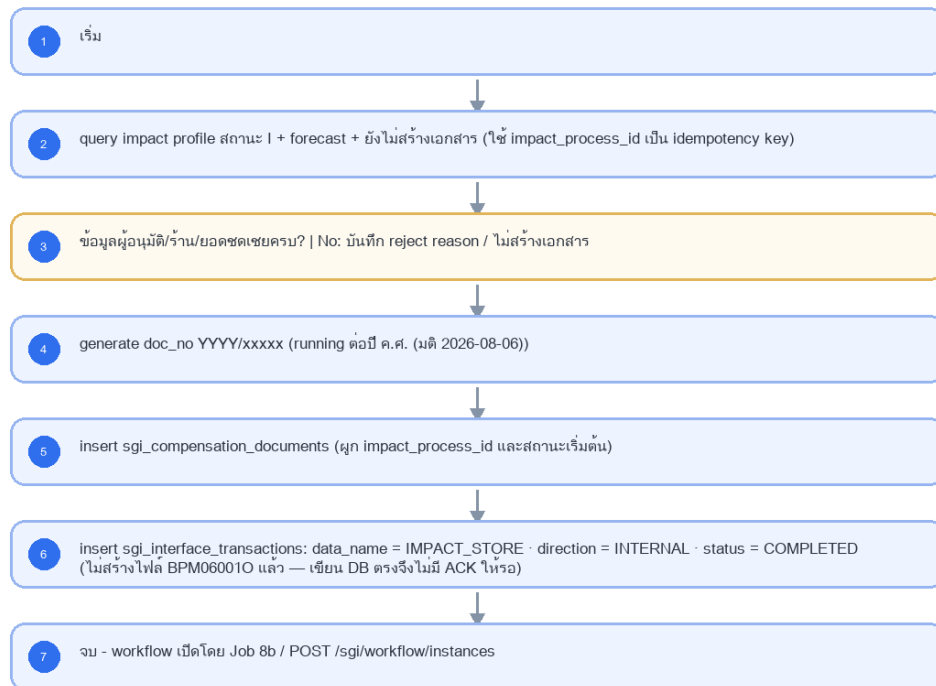
2.8 Job 8 Document Number Gap and Rerun Policy

Job 8 ใช้ running number แบบ monotonic ต่อปี ค.ศ. ช่องว่างของเลขเอกสารจาก concurrent rerun หรือ ON CONFLICT เป็นพฤติกรรมที่ยอมรับได้ เพราะเลขที่มีหน้าที่รับประกัน uniqueness ไม่ได้รับประกันความต่อเนื่อง

Case	Required behavior	Evidence / metric
Rerun พบ impact_process_id เดิมก่อนจองเลข	คืน/ข้ามด้วย doc_no เดิมโดยไม่จอง running_no เพิ่มเมื่อ fast lookup พบข้อมูลแล้ว	duplicateExistingCount + existingDocNo
Concurrent worker ชน ON CONFLICT หลังจองเลข	ยอมให้ running_no ที่จองแล้วกลายเป็น gap; ห้ามลด sequence และห้ามนำเลขกลับมาใช้	numberGapCount + conflictedImpactProcessId
Conflict path	อ่าน sgi_compensation_documents ด้วย impact_process_id แล้วใช้ d.doc_no เดิมสำหรับ tracking/reconcile	tracking.doc_no ตรงกับเอกสารที่ commit อยู่จริง
New document path	insert document และ tracking (direction=INTERNAL) ใน transaction เดียว	createdCount และ trackingCount เพิ่มเท่ากัน
Audit/runbook	อธิบายว่าเลขอาจไม่ต่อเนื่องแต่ต้องไม่ซ้ำและตรวจสอบย้อนกลับได้	ไม่มีขั้นตอน manual reuse หรือ renumber

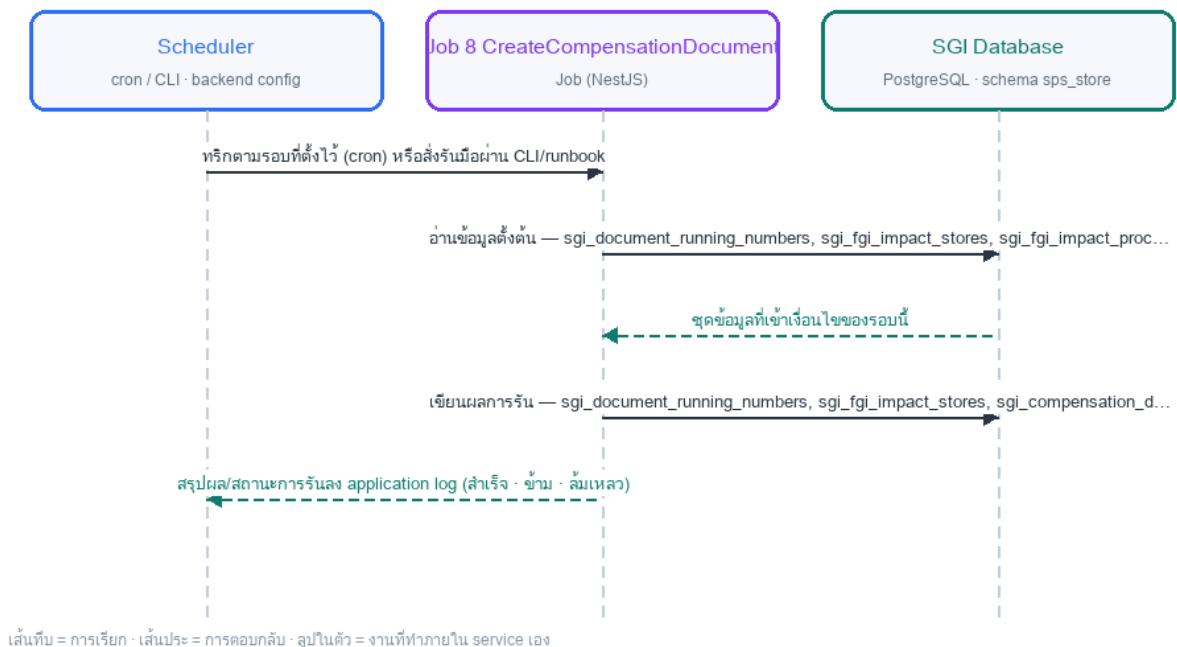
3. Flow และ Sequence Diagram

LLDD BE - Job 8 CreateCompensationDocument



รูปที่ 1: Implementation flow: LLDD BE - Job 8 CreateCompensationDocument

Sequence — LLDD BE - Job 8 CreateCompensationDocument



รูปที่ 2: Sequence diagram: LLDD BE - Job 8 CreateCompensationDocument

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_document_running_numbers	R/W	ตัวนับเลขเอกสารรายปี — ออก doc_no YYYY/xxxx (ค.ศ.)
sgi_fgi_impact_stores	R/W	อ่าน candidate และอัปเดตสถานะสร้างเอกสาร
sgi_fgi_impact_processes	R	hub รอบชดเชย
sgi_compensation_documents	W	สร้างหัวเอกสารแทนไฟล์ BPM06001O
sgi_interface_transactions	W	tracking ภายใน: direction=INTERNAL · status=COMPLETED (ไม่มี ACK ให้รอเพราะเขียน DB ตรง)

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	sgi-job-8 ใน SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts
service ของ job	SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-8-*.service.ts
เทสของ job	SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-8-*.spec.ts
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก *.sql.ts)
คำอธิบายภาษาคนของ job	batchjob/JOB-08-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน INPUT (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ @Cron ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนฟัง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (BATCH_START/BATCH_END + runId — src/main.ts ทำให้แล้ว) และบันทึก integration_log อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

6 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · • เทสที่ mock query runner ต้องป้อนรูป tuple [rows, rowCount] สำหรับ UPDATE/DELETE (กฎ #140)

การกันสร้างเอกสารซ้ำ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	รอบนี้มีเอกสารแล้ว	มีแถวที่ impact_process_id เดียวกัน	skip และคืน doc_no เดิม ไม่สร้างใหม่

ความพร้อมของรอบ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-02	รอบยัง active	flag_action IN ('Y', 'W')	สร้างเอกสารต่อได้
UT-03	รอบไม่ active	flag_action เป็นค่าอื่น	ไม่สร้างเอกสาร

เลขเอกสาร

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-04	ออกเลขใหม่	ปี 2026 · running_no ล่าสุด 122	ได้ 2026/00123 — ปี ค.ศ. 5 หลัก zero-pad
UT-05	ออกเลขพร้อมกัน 2 รอบ	สองทรานแซกชันขอเลขปีเดียวกัน	lock running number ต่อปีในทรานแซกชันเดียวกับการสร้างเอกสาร → ไม่ได้เลขซ้ำ
UT-06	อ่านผลของ UPDATE ... RETURNING	TypeORM คืน [rows, rowCount]	ต้องอ่านผ่าน queryFirstRow ไม่ใช่ rows[0] (กฎ #140) ไม่งั้นออกเลขไม่ได้เลย

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportImpactStoreFlowToBPM.java — lines 9-17

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/main/ExportImpactStoreFlowToBPM.java
Original lines	9-17
Responsibility	Legacy main entrypoint for exporting impact-store flow data.

```
public class ExportImpactStoreFlowToBPM {
    private static ApplicationContext context = new FileSystemXmlApplicationContext("config.xml");
    private static ExportController exportController = (ExportController) context.getBean("exportController");
    public static void main(String[] args){
        LogUtils.info(ExportImpactStoreFlowToBPM.class,"Start => export IMPACT_STORE_INFO to BPM ");
        exportController.exportImpactStoreFlowToBPM();
        LogUtils.info(ExportImpactStoreFlowToBPM.class,"End => export IMPACT_STORE_INFO to BPM");
    }
}
```

J2. ExportController.java — lines 518-529

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	518-529
Responsibility	Build impact-store BPM payload, write file, upload, backup, notification.

```
public void exportImpactStoreFlowToBPM() {
    String fileName = "";
    Calendar calStart = Calendar.getInstance(Locale.US);
    EmailTemplateInformStatusBean informBean = new EmailTemplateInformStatusBean();
    informBean.setSystemCode(FgiConstant.SYSTEM_CODE);
    informBean.setJobName(FgiConstant.JOB_NAME_EXPORT_IMPACT_STORE_FLOW_TO_BPM);
    informBean.setStartDateTime(calStart.getTime());
    informBean.setImportFileName("");
    List<Map> errorProcessListMap = new ArrayList<>();
    try {
        fileInputStream = th.co.gosoft.fcs.utils.FileUtils.openFile("ApplicationResources.properties");
        propertiesFile.load(fileInputStream);
    }
```

J2. ExportController.java — lines 646-657

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/controller/ExportController.java
Original lines	646-657
Responsibility	Build impact-store BPM payload, write file, upload, backup, notification.

```
LogUtils.info(getClass(),"--Transaction is Rollback--");
LogUtils.error(getClass(),ex);
informBean.setErrMsg(""+ ex);
informBean.setStatus(FgiConstant.JOB_STATUS_FAIL);
return new Boolean("false");
}
});
if(!result){
return "";
}
return fileName;
}
```

J3. ExportJdbc.java — lines 1654-1665

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1654-1665
Responsibility	Query impact-store rows eligible for workflow export.

```
public List<Map> queryImpactStoreToBPM(){ // to BPM
StringBuilder sql = new StringBuilder();
sql.append(" select si.impact_store_info_id, si.storecode_i, si.name_i, si.opendate_i, si.zone_i, ");
sql.append(" si.branchtype_i, si.juristic_id_i, si.juristic_name_i, si.url_all_map, si.impact_type, ");
sql.append(" si.dv_emp_id, si.dv_fname_th, si.dv_lname_th, si.dv_fname_en, si.dv_lname_en, si.dv_email, ");
sql.append(" si.gm_emp_id, si.gm_fname_th, si.gm_lname_th, si.gm_fname_en, si.gm_lname_en, si.gm_email, ");
sql.append(" si.avp_emp_id, si.avp_fname_th, si.avp_lname_th, si.avp_fname_en, si.avp_lname_en, si.avp_email, ");
sql.append(" si.SBP_DV_EMAIL1, si.SBP_DV_EMP_ID1, si.SBP_DV_FNAME_EN1, ");
sql.append(" si.SBP_DV_FNAME_TH1, si.SBP_DV_LNAME_EN1, si.SBP_DV_LNAME_TH1, si.SBP_GM_EMAIL1, si.SBP_GM_EMP_ID1, ");
sql.append(" si.SBP_GM_FNAME_EN1, si.SBP_GM_FNAME_TH1, si.SBP_GM_LNAME_EN1, si.SBP_GM_LNAME_TH1, ");
sql.append(" si.SBP_VP_EMAIL1, si.SBP_VP_EMP_ID1, si.SBP_VP_FNAME_EN1, si.SBP_VP_FNAME_TH1, ");
sql.append(" si.SBP_VP_LNAME_EN1, si.SBP_VP_LNAME_TH1, ");
```

J3. ExportJdbc.java — lines 1681-1692

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	1681-1692
Responsibility	Query impact-store rows eligible for workflow export.

```
sql.append(" and not exists ( ");
sql.append(" select rd.transaction_pk ");
sql.append(" from fgi_confirm_receive_data rd ");
sql.append(" where rd.data_name = 'IMPACT_STORE' ");
sql.append(" and rd.transaction_pk = si.impact_store_info_id ");
sql.append(" and rd.month = si.compensate_month ");
sql.append(" and rd.year = si.compensate_year ");
sql.append(" ) ");
List<Map> impactStoreList = jdbcTemplate.queryForList(sql.toString());
LogUtils.info(getClass(),"query FGI_IMPACT_STORE_INFO for export : "+impactStoreList.size());
return impactStoreList;
}
```